CS780 Capstone Project: OBELIX: the Warehouse Robot

FirstName <Middle Name if any> LastName
roll no.
{name}@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This report studies the OBELIX capstone task, where a robot must find, attach to, push, and unwedge a box under partial observability. I modeled the problem as a POMDP and evaluated several controllers, including DDQN, symmetric PPO, handcrafted phase logic, and asymmetric PPO. The final solution uses two specialized PPO actors—one for wall settings and one for no-wall settings—together with an observation-only switch. The main lesson is that strong performance required explicit handling of partial observability, asymmetric training, and different behaviors for wall and no-wall regimes.

1 Competition Result

Codabench Username: _____
Final leaderboard rank (Test Phase): _____
Leaderboard rank – Level 1 (static): _____
Leaderboard rank – Level 2 (blinking): _____
Leaderboard rank – Level 3 (moving + blinking): _____

Number of valid submissions made (Development(Level 1, Level 2, Level 3) and Test Phase):

Phase	Number of submissions
Development – Level 1	___
Development – Level 2	___
Development – Level 3	___
Test Phase	___

2 Introduction and Problem Description

The OBELIX task is a sequential robot-control problem in which the agent must first find a grey box, then make contact, push it to the arena boundary, and finally unwedge it so that the box leaves the workspace. The robot has only five discrete actions:

$$\mathcal{A} = \{\texttt{L45}, \texttt{L22}, \texttt{FW}, \texttt{R22}, \texttt{R45}\}.$$

The first four actions rotate the robot by fixed angles, while `FW` attempts a forward move. Once the box is attached, forward motion attempts to move both the robot and the box.

The environment becomes progressively harder across the three levels. In Level 1 the box is static. In Level 2 the box blinks, so the robot may temporarily lose it and cannot attach while it is invisible. In Level 3 the box both blinks and moves before contact. In addition, a vertical wall with a narrow opening may appear, which changes the problem from simple pushing to constrained navigation and makes wrong-side contact especially costly. The high-level objective is to maximize mean cumulative reward across these settings while remaining robust to wall obstacles.

A natural formalization is a partially observable Markov decision process

$$\mathcal{P} = (\mathcal{S}, \mathcal{A}, \Omega, P, O, r, \rho_0, \gamma, H),$$

where $s_t \in \mathcal{S}$ is the latent simulator state, $a_t \in \mathcal{A}$ is the chosen action, $o_t \in \Omega$ is the legal observation, $P(s_{t+1} \mid s_t, a_t)$ is the transition model, $O(o_t \mid s_t)$ is the observation model, r is the scalar reward, ρ_0 is the initial-state distribution, γ is the discount factor, and H is the finite horizon. Although the simulator is Markov, the deployed agent does not observe the full state, so the control problem is genuinely partially observable.

The legal observation is an 18-bit sensor vector

$$o_t = [b_0, \ldots, b_{15}, b_{\mathrm{IR}}, b_{\mathrm{stuck}}] \in \{0,1\}^{18}.$$

The first 16 bits are sonar-style sector indicators, b_{IR} indicates that the box is directly ahead, and b_{stuck} indicates that the robot is jammed against a wall or boundary. Because many different latent states can generate the same observation, a memoryless reactive policy is often insufficient. The correct control law is therefore history-dependent:

$$\pi(a_t \mid h_t), \qquad h_t = (o_0, a_0, o_1, a_1, \ldots, o_t).$$

This partial observability is the main reason the task is difficult: the agent must infer whether it is searching, approaching, already attached, badly aligned, or stuck from short observation histories rather than from explicit phase labels.

The reward structure is also unusually conservative. Each step carries a cost of -1, repeated stuck states incur a large penalty, and success requires many coordinated actions before the final bonus is obtained. A compact summary of the environment reward is

$$r_t = -1 - 200\,\mathbf{1}\{b_{\mathrm{stuck}} = 1\} + r_{\mathrm{sensor}} + r_{\mathrm{attach}} + r_{\mathrm{success}}.$$

The one-time sensor bonuses reward discovering informative geometry, first attachment yields $+100$, and successful removal of the box gives a terminal bonus of $+2000$. This creates a hard credit-assignment problem: a locally harmless rotation sequence may be necessary for long-run success, while one poorly timed forward move can immediately produce a -201 transition. During training I therefore used a tensorized implementation of the environment for high-throughput rollout collection, but all final evaluation logic was checked against the reference evaluator.

3 Approach and Model Evolution

3.1 Chronological model evolution

The project did not converge to the final agent in one step. The main sequence of experiments was: DDQN, symmetric PPO, handcrafted phase controllers for diagnosis, asymmetric PPO with privileged critics, memory-augmented actors, and finally an observation-only switch between wall-specialized and no-wall-specialized PPO policies.

The first serious attempt was DDQN because the action space is small and discrete. In principle, learning $Q_\theta(h_t, a_t)$ with replay and target networks should have been enough. In practice, it failed for a structural reason: the same local 18-bit observation could correspond to "forward is useful" or "forward is catastrophic" depending on hidden wall geometry, box visibility, and prior motion. The resulting bootstrapped targets were extremely noisy, especially when stuck penalties dominated otherwise harmless turn transitions. Replay increased sample count but did not resolve state aliasing.

The next stage was symmetric PPO. This was more stable than DDQN because it optimized a stochastic policy directly instead of relying on fragile max-Q targets. It learned viable search behavior in easier settings, but wall cases were still unreliable. The agent often oscillated near contact, approached from the wrong side, or entered repeated stuck loops. To understand these failures, I temporarily built handwritten phase-based controllers. These were not competitive final policies, but they were useful diagnostically because they clarified that the task truly contains different latent modes: search, attach, and push/unwedge.

This insight led to asymmetric PPO. The key idea was to let the actor observe only legal deployment features while allowing the critic to use privileged simulator information during training. That made value estimation easier without violating the rules at inference time. I also tested recurrent memory, relative-pose features, teacher transfer, routers, distillation, and macro-assisted policies. Some of these experiments produced impressive local scores, but many of them were brittle on unseen seeds. The final selected strategy was therefore the simplest robust one: two specialized PPO branches and a legal observation-only switch.

3.2 Observation engineering

The no-wall branch used a compact 32-dimensional legal encoding. Let the raw observation be

$$o_t = [b_0, \ldots, b_{15}, b_{\mathrm{IR}}, b_{\mathrm{stuck}}] \in \{0,1\}^{18}.$$

The first 16 bits are sonar detections, partitioned as left sector $b_{0:3}$, front sector $b_{4:11}$, and right sector $b_{12:15}$. Bit b_{IR} indicates direct frontal contact and b_{stuck} indicates that the robot is jammed. The no-wall actor did not use these 18 bits alone; instead it formed

$$x_t = [o_t, \phi(o_t)] \in \mathbb{R}^{32},$$

where $\phi(o_t) \in \mathbb{R}^{14}$ is a deterministic feature map computed from the legal observation only. The exact 32-dimensional encoding is shown in Table **??**.

Indices	Name	Definition
0:3	left sonar bits	Raw binary sonar indicators for the four left-facing sectors.
4:11	front sonar bits	Raw binary sonar indicators for the eight front-facing sectors.
12:15	right sonar bits	Raw binary sonar indicators for the four right-facing sectors.
16	IR/contact bit	Raw frontal contact-style indicator.
17	stuck bit	Raw jam/stuck indicator.
18	left_active	$\mathbf{1}\{\sum_{i=0}^{3} b_i > 0\}$.
19	front_active	$\mathbf{1}\{\sum_{i=4}^{11} b_i > 0\}$.
20	right_active	$\mathbf{1}\{\sum_{i=12}^{15} b_i > 0\}$.
21	IR copy	Direct copy of b_{IR}.
22	stuck copy	Direct copy of b_{stuck}.
23	blind	$\mathbf{1}\{\sum_{i=0}^{15} b_i = 0\}$, indicating that no sonar sector is active.
24	left_count	$\sum_{i=0}^{3} b_i$.
25	front_count	$\sum_{i=4}^{11} b_i$.
26	right_count	$\sum_{i=12}^{15} b_i$.
27	front_far_count	Sum of front bits at offsets $0, 2, 4, 6$ within the front block, i.e. global indices $4, 6, 8, 10$.
28	front_near_count	Sum of front bits at offsets $1, 3, 5, 7$ within the front block, i.e. global indices $5, 7, 9, 11$.
29	left-minus-right	left_count − right_count.
30	right-minus-left	right_count − left_count.
31	front-minus-side-mean	front_count − $\frac{1}{2}$(left_count + right_count).

Table 1: Exact 32-dimensional legal observation encoding used by the no-wall PPO branch. The first 18 entries are raw observations and the final 14 entries are derived deterministically from those same legal observations.

This representation was sufficient in the no-wall setting because the main difficulty there was search under blinking and motion, not fine obstacle negotiation. The derived features summarize whether the robot is blind, whether detections are concentrated in the front or on one side, and whether the front pattern is near-dominant or far-dominant. In practice, this made the no-wall policy easier to optimize than using the 18-bit vector alone.

The wall setting required more memory than the 32-dimensional feedforward encoding. For that branch, a single observation was usually insufficient to distinguish blind search from repeated contact or a persistent jam. I therefore used a larger legal recurrent feature vector built from a short stack of recent observations, recent actions, and counters derived from legal sensor histories only. These recurrent features included observation history, action history, blind/contact/stuck counters, seen-side memory, and wall-hit memory inferred through dead reckoning rather than from true simulator pose. This gave the actor enough temporal context to reason about short-term motion patterns without reading hidden simulator variables.

3.3 Why PPO and why asymmetric training

Among the methods discussed in the course, PPO was the most practical choice for this project. A tabular method is not appropriate because the latent state is continuous and hidden, and even the legal observation history is too large to tabulate. Pure Monte Carlo control is also unattractive because episodes are long and successful trajectories are sparse, which makes full-return updates high variance. PPO instead uses a learned critic and generalized advantage estimation (GAE),

giving a useful middle ground between high-variance Monte Carlo targets and unstable off-policy value learning.

The training setup was asymmetric:

$$\pi_\theta(a_t \mid x_t), \qquad V_\psi(x_t, p_t),$$

where x_t is the legal actor input and p_t is privileged simulator information available only to the critic during training. PPO then optimizes the clipped objective

$$L^{\text{clip}}(\theta) = \mathbb{E}\Big[\min\Big(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big],$$

with GAE-based advantages

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}.$$

This improved critic quality, reduced variance in policy updates, and kept the deployed actor fully legal.

4 Experiments and Implementation Details

4.1 Training setup

To make PPO training practical, I implemented a tensorized training backend that replicated the reference environment logic in batched torch form. This allowed rollout collection from hundreds of environments at once on CUDA while preserving parity with the original evaluator. Typical rollouts used $N \in \{256, 512\}$ vectorized environments and rollout length $L = 128$. The large rollout batches reduced seed-to-seed noise and made ablations practical.

The final wall branch was trained with asymmetric PPO, while the no-wall branch was trained with standard PPO on the vectorized backend. The main hyperparameters are summarized in Table **??**. During candidate screening, all models were compared under the same local protocol: difficulty 3, horizon $T = 2000$, ten runs, and base seed 0. This was important because the environment has high variance, and comparing checkpoints under different horizons or seed ranges can be misleading.

Hyperparameter	Wall branch	No-wall branch
Algorithm	Asymmetric PPO	PPO
Actor input dimension	271	32
Actor architecture	$271 \to 512 \to 256 \to 128 \to 5$	$32 \to 128 \to 64 \to 5$
Critic architecture	$1024 \to 512 \to 256$	standard actor-critic
Vectorized environments	512	256
Rollout length	128	128
Discount factor γ	0.995	0.995
GAE λ	0.95	0.95
Learning rate	1.5×10^{-4}	2×10^{-4}
Entropy coefficient	0.002	0.005
Total environment steps	10^6 update-scale training	4,194,304

Table 2: Main hyperparameters of the final two PPO branches.

4.2 Final inference policy

The final submission was not a single monolithic network. Instead, it used two actors combined through a lightweight observation-only switch. For the first 80 steps, the agent ran the wall branch as a probe and accumulated three legal statistics:

$$C = \text{contact-like front-near counts}, \qquad B = \text{blind observations}, \qquad F = \text{front hits}.$$

At the end of the probe window, it switched to the no-wall branch when

$$C \leq 15, \qquad B \geq 40,$$

and otherwise stayed with the wall branch. Empirically, no-wall starts tended to produce long blind-search segments with little early contact, while wall starts produced more early side/front interactions and stuck-like patterns. This simple switch was more reliable than the later learned routers.

At inference time, both branches used only legal observations and history features derived from those observations and the policy's own past actions. Privileged information was strictly training-only. This was essential because the capstone rules do not allow the deployed agent to inspect hidden simulator state.

5 Results

Table **??** summarizes the most important development stages. The progression shows that the project improved not by making a single model larger, but by correcting the mismatch between algorithm and problem structure.

Stage	Main idea	Outcome
DDQN	Off-policy value learning with replay and target network	Unstable under observation aliasing and large stuck penalties
Symmetric PPO	Single stochastic actor-critic from legal observations only	Better search behavior, but poor wall robustness
Handwritten phase logic	Explicit search/attach/push routines	Useful diagnosis, but brittle and not general enough
Asymmetric PPO	Legal actor with privileged critic during training	Major improvement in value estimation and stability
Switch policy	Separate wall and no-wall experts routed by legal early observations	Best balanced and most robust final design

Table 3: Most important development stages and their practical outcome.

The main local evaluation for the final clean switch submission is shown in Table **??**. The no-wall branch was clearly strong, while the wall branch remained the main bottleneck. The weighted score was positive, but the large standard deviations already indicated that hidden-seed robustness would matter more than a single mean value.

Condition	Mean return	Std. dev.	Runs
No wall, difficulty 3	1820.4	1539.7	10
Wall, difficulty 3	-1142.8	1846.5	10
Weighted score $0.4R_{\mathrm{nw}} + 0.6R_{\mathrm{w}}$	42.48	–	–

Table 4: Local evaluation of the final clean switch submission on ten seeds.

I also evaluated several late-stage candidates that added macros or learned routers. Their local scores are shown in Table **??**. Although some of them achieved much higher local weighted scores, they did not transfer reliably to hidden seeds. For that reason, the final selected model prioritized robustness and legality over the highest local proxy score.

Candidate	Wall	No-wall	Weighted	Comment
Clean switch baseline	-1142.8	1820.4	42.48	balanced local baseline
Macro-assisted wall policy	51.6	1252.9	532.1	strong locally, likely overfit
Learned-router student	51.6	1827.5	761.64	high proxy score, weak hidden transfer
Random-probe router	-865.7	1363.3	25.9	inconsistent
Guarded conservative router	-1481.7	325.9	-758.7	too passive

Table 5: Late-stage local candidates. Higher proxy score did not necessarily imply better hidden-seed performance.

Overall, the best-performing design was the two-branch PPO system with legal switching. The main reason was not just model capacity. It matched the problem geometry better: the no-wall regime rewards efficient search and direct pushing, whereas the wall regime rewards short-memory obstacle reasoning and conservative alignment before forward motion.

6 Error Analysis and Discussion

The final system worked better than the earlier attempts because it addressed the three dominant structural issues in the task. First, it treated OBELIX as a POMDP rather than as a purely reactive sensor-action mapping. Second, it separated actor information from critic information through asymmetric training, which improved advantage quality without leaking hidden state at deployment. Third, it did not force a single behavior to solve both wall and no-wall settings.

Even with that design, the wall regime remained difficult. The wall setting combines all the adverse factors at once: partial observability, blinking or moving targets, narrow geometry, and severe penalties for repeated contact. In these states, the value of `FW` can change sign with a small change in hidden alignment. A locally plausible forward move may create another jam, while a short rotation sequence that looks unproductive can be necessary for a good push angle.

The main failure modes were different across difficulty levels. In Level 1, errors usually came from wrong-side contact or passive turning policies that delayed useful attachment even though the box was static. In Level 2, blinking caused loss of track; the agent sometimes committed to a stale approach direction and then wandered after the box disappeared. In Level 3, the moving box introduced interception errors: the robot could detect the box, but still approach too late or from the wrong angle. With walls, the most common failure was a collision loop in which the robot attached badly, pushed into the obstacle, and repeatedly triggered stuck penalties.

A second important finding was methodological: strong local scores were not a reliable indicator of hidden-seed performance. Macro-assisted policies and learned routers could fit the local proxy seeds extremely well, but those gains often vanished on unseen runs. The safest conclusion is that OBELIX is not just a score-maximization problem; it is a robustness problem under partial observability. With more time, I would focus on better belief-state modeling, stronger recurrent or transformer-style memory, explicit curricula for wall alignment, and more principled switching or option-learning methods.

7 Conclusion

LLM Usage Declaration

- Which Large Language Model (LLM) did you use to complete the competition or write the report?

- In which part did you use the LLM, and how did it help you?

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts		
Ideation / brainstorming		
Debugging small code snippets		
Plotting / visualization code		
Grammar / writing style		
Report structure / section ideas		
Other (specify)		

- Did you refer to any other sources or websites apart from the LLM and lecture slides?
  If yes, please mention clearly (papers, blogs, GitHub repos, textbooks etc.)

- Apart from any LLM usage, what was your own contribution?
  Briefly describe your personal understanding, reasoning, implementation effort, analysis, and experimentation.